



DEZYNE ECOLE COLLEGE

PROJECT REPORT

ASSOCIATION RULE DATA MINING *Project*



Based on student enrollment patterns
in programming courses



SUBMITTED BY

Mohit Tanwar



DEPARTMENT

BCA (Bachelor of Computer Applications)



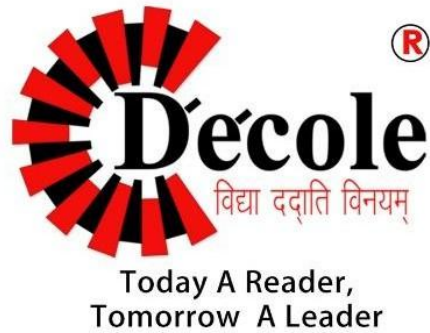
YEAR

3rd Year VI sem



COLLEGE

Dezyne École College



Dezyne École College
www.dezyneecole.com

Title of Project

Association Rule Mining Project

Based on student enrolment Patterns

Subject Name

Data Mining with R

Submitted By

Mohit Tanwar

Department – BCA

Semester – 6th

Session – 2023-2026

Project Report of
Association Rule Mining Project
Based on student enrollment Patterns

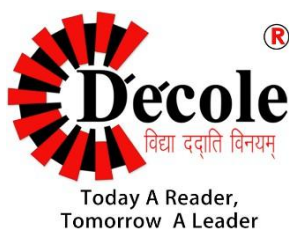
Submitted to

Dezyne École College
Towards the Partial Fulfilment of

6th
Semester

By

Mohit Tanwar



www.dezyneecole.com

2024-2025

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Mohit Tanwar

Department of BCA

Dezyne École College, Ajmer

TABLE OF CONTENTS

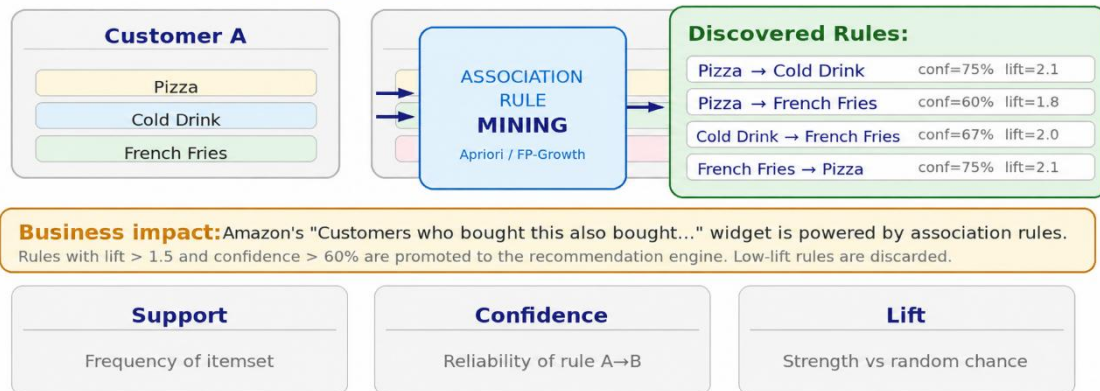
Contents

Data Mining with R.....	2
Project Report of.....	3
Submitted to	3
6th.....	3
Mohit Tanwar	3
Acknowledgement.....	4
Section 1: Introduction — What is Market Basket Analysis?	7
1.1 Key Definitions	7
Section 2: Practical Example 1 — Grocery Store (10 Transactions)	9
2.1 The Transaction Database.....	9
2.2 Step 1 — Count Individual Item Frequencies	9
2.3 Step 2 — Generate and Count 2-Item Candidates.....	10
Section 3: The Three Key Metrics — Support, Confidence, Lift	12
3.1 Support	12
3.2 Confidence.....	12
3.3 Lift — The Most Important Metric.....	13
Section 4: The Apriori Algorithm — Full Step-by-Step Walkthrough	15
4.1 The Core Principle	15
4.2 Complete Algorithm with Our Example	15
4.3 Rule Generation from Frequent Itemsets.....	16
4.4 Time & Space Complexity	17

Section 5: All Mined Rules — Confidence, Lift & Amazon Decision	18
5.1 Complete Rules Table for Grocery Example	18
Section 6: Practical Example 2 — Amazon Electronics (5 Transactions).....	20
6.1 Transaction Database	20
6.2 1-Itemset Frequencies.....	21
6.3 Top Rules with High Lift.....	21
Section 7: Practical Example 3 — Manual Apriori (Exam-Style)	23
7.1 The Given Database	23
7.2 Pass 1 — Frequent 1-Itemsets	23
7.3 Pass 2 — Frequent 2-Itemsets	24
7.4 Pass 3 — Frequent 3-Itemsets	24
7.5 Rule Generation (MinConf = 75%)	25
Section 8: FP-Growth vs Apriori — Comparison.....	27
8.1 FP-Tree Construction.....	27
8.2 FP-Tree for Our Grocery Example	27
8.3 Comparison Table	28
Section 9: Industry Application — How Amazon Uses Association Rules	29
9.1 Where Association Rules Appear on Amazon	29
9.2 The Full Amazon Pipeline	29
Section 10: Summary — Formulas, Pitfalls & Exam Tips.....	31
10.1 Complete Formula Reference	31
10.2 Apriori Algorithm Summary	31
10.3 Common Exam Questions & Answers	31

Section 1: Introduction — What is Market Basket Analysis?

Market Basket Analysis — How it works



Assignment A1 — Amazon Frequently Bought Together | Data Mining | BCA

The Business Problem

Amazon processes hundreds of millions of transactions every day. Hidden inside these purchases are patterns: customers who buy a laptop almost always buy a mouse. Customers who buy bread frequently buy butter. If Amazon can discover these patterns automatically, it can display a 'Frequently Bought Together' recommendation panel — increasing average order value without any human curation.

Association Rule Mining is the algorithmic technique that discovers these patterns. Given a database of transactions, it finds rules of the form:

$A \rightarrow B$ (If a customer buys A, they are likely to also buy B)

and assigns each rule three scores that measure how useful it is.

1.1 Key Definitions

Term	Formal Definition	Intuitive Meaning	Example
Transaction	A single shopping session with a set of items purchased	One customer's cart	$T1 = \{\text{Pizza, Cold Drink, French Fries}\}$
Itemset	Any subset of items	A group of products	$\{\text{Pizza, French Fries}\}$

Frequent Itemset	An itemset whose support $\geq$ minimum threshold	Appears in enough baskets	{ Pizza, French Fries } in 4/10 baskets
Association Rule	An implication $A \rightarrow B$ derived from transactions	If-then pattern	Pizza $\rightarrow$ French Fries
Support	Fraction of transactions containing the itemset	How common is this combination?	support = $4/10 = 40\%$
Confidence	Fraction of A-transactions that also contain B	How reliable is the rule?	conf = $4/6 = 67\%$
Lift	Confidence divided by support of B	Is the rule better than random?	lift = $0.67/0.60 = 1.11$

Section 2: Practical Example 1 — Grocery Store (10 Transactions)

2.1 The Transaction Database

Setup: A grocery store records 10 customer baskets. Each basket is one transaction. We set minimum support = 40% (an itemset must appear in at least 4 out of 10 transactions to be considered 'frequent') and minimum confidence = 60%.

Transaction	Items Purchased	No. of Items
T01	{Pizza, Cold Drink, French Fries}	3
T02	{Pizza, French Fries, Burger}	3
T03	{Cold Drink, Burger, Pasta}	3
T04	{Pizza, Cold Drink, French Fries, Burger}	4
T05	{Pizza, Cold Drink}	2
T06	{French Fries, Burger, Pasta}	3
T07	{Pizza, French Fries, Burger, Cold Drink}	4
T08	{Cold Drink, Pasta, Ice Cream}	3
T09	{Pizza, Burger, Ice Cream}	3
T10	{French Fries, Cold Drink, Pasta, Ice Cream}	3

2.2 Step 1 — Count Individual Item Frequencies

Scan all 10 transactions and count how many times each item appears. Compare with the minimum support threshold (4 occurrences = 40%).

Item	Appears in transactions	Count	Support	Frequent?
------	-------------------------	-------	---------	-----------

Pizza	T01,T02,T04,T05,T07,T09	6	6/10 = 60%	YES ✓
Cold Drink	T01,T03,T04,T05,T07,T08,T10	7	7/10 = 70%	YES ✓
French Fries	T01,T02,T04,T06,T07,T10	6	6/10 = 60%	YES ✓
Burger	T02,T03,T04,T06,T07,T09	6	6/10 = 60%	YES ✓
Pasta	T03,T06,T08,T10	4	4/10 = 40%	YES ✓
Ice Cream	T08,T09,T10	3	3/10 = 30%	NO — pruned ✗

Apriori Principle applied: Ice Cream has support 30% < 40% threshold. This means we NEVER need to check any 2-itemset containing Ice Cream — {Pizza, Ice Cream}, {Cold Drink, Ice Cream}, etc. are all automatically infrequent. We prune them without even computing their support. This is the power of the Apriori Principle.

2.3 Step 2 — Generate and Count 2-Item Candidates

Generate all pairs from the 5 surviving frequent items (Pizza, Cold Drink, French Fries, Burger, Pasta). That is $C(5,2) = 10$ candidate pairs.

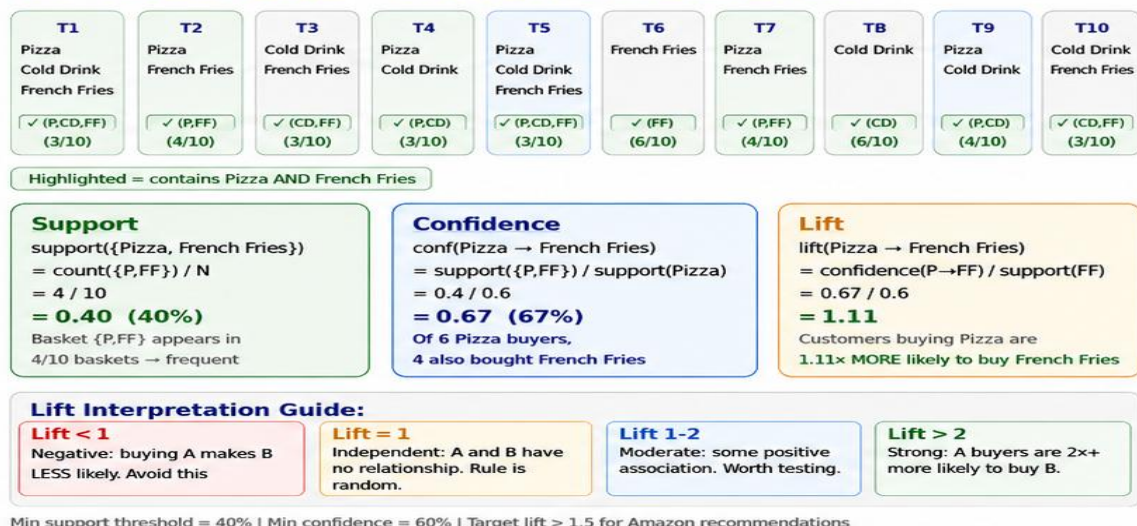
2-Itemset	Transactions	Count	Support	Frequent?
{Pizza, Cold Drink}	T01,T04,T05,T07	4	40%	YES ✓
{Pizza, French Fries}	T01,T02,T04,T07	4	40%	YES ✓
{Pizza, Burger}	T02,T04,T07,T09	4	40%	YES ✓
{Cold Drink, French Fries}	T01,T04,T07,T10	4	40%	YES ✓
{Cold Drink, Burger}	T03,T04,T07	3	30%	NO ✗ — pruned
{Cold Drink, Pasta}	T03,T08,T10	3	30%	NO ✗ — pruned
{French Fries, Burger}	T02,T04,T06,T07	4	40%	YES ✓
{French Fries, Pasta}	T06,T10	2	20%	NO ✗ — pruned

{Burger, Pasta}	T03,T06	2	20%	NO X — pruned
{Pizza, Pasta}	T00 (none)	0	0%	NO X — pruned

Result: 5 frequent 2-itemsets: {Pizza,Cold Drink}, {Pizza,French Fries}, {Pizza,Burger}, {Cold Drink,French Fries}, {French Fries,Burger}.

Section 3: The Three Key Metrics — Support, Confidence, Lift

The Three Metrics — Support, Confidence, Lift



3.1 Support

Formula: $\text{support}(A \cup B) = \text{count}(A \cup B) / N$

Meaning: How often does the combination {Netflix, Amazon Prime Video} appear across ALL transactions? A higher support means a more statistically significant pattern.

Example: {Pizza, French Fries} appears in T01, T02, T04, T07 → count = 4 → support = 4/10 = 40%

Threshold: We set minSupport = 40%. Any itemset below this is too rare to be meaningful — it might be a coincidence rather than a pattern.

- If support is TOO LOW: the rule is based on too few transactions (statistically unreliable).
- If support is TOO HIGH: only very obvious rules survive (e.g. {Pizza, Cold Drink} — not useful for recommendations).
- In practice, Amazon uses minSupport of 1–5% because they have billions of transactions.

3.2 Confidence

Formula: $\text{confidence}(A \rightarrow B) = \text{support}(A \cup B) / \text{support}(A)$

Meaning: Of all customers who bought A, what percentage also bought B? This measures the **RELIABILITY** of the rule 'A causes B'.

Example — Pizza→French Fries:

- $\text{support}(\{\text{Pizza}, \text{French Fries}\}) = 4/10 = 0.40$
- $\text{support}(\{\text{Pizza}\}) = 6/10 = 0.60$
- $\text{confidence} = 0.40 / 0.60 = 0.667 = 67\% \rightarrow$ Of the 6 people who bought Pizza, 4 also bought French Fries.

Example — French Fries→Pizza:

- $\text{support}(\{\text{Pizza}, \text{French Fries}\}) = 4/10 = 0.40$
- $\text{support}(\{\text{French Fries}\}) = 6/10 = 0.60$
- $\text{confidence} = 0.40 / 0.60 = 0.667 = 67\% \rightarrow$ Same value here, but not always symmetric!

Important: Confidence is NOT symmetric. $\text{conf}(A \rightarrow B) \neq \text{conf}(B \rightarrow A)$ in general.

3.3 Lift — The Most Important Metric

Formula: $\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$

Meaning: Is the association **STRONGER** than what you'd expect by random chance? Lift corrects for the natural popularity of B.

Why confidence alone misleads: Cold Drink appears in 70% of all transactions. A rule $X \rightarrow \text{Cold Drink}$ might have 70% confidence simply because Cold Drink is popular — nothing to do with X. Lift divides by $\text{support}(\text{Cold Drink}) = 0.70$ to correct for this baseline.

Lift = 1 EXAMPLE:

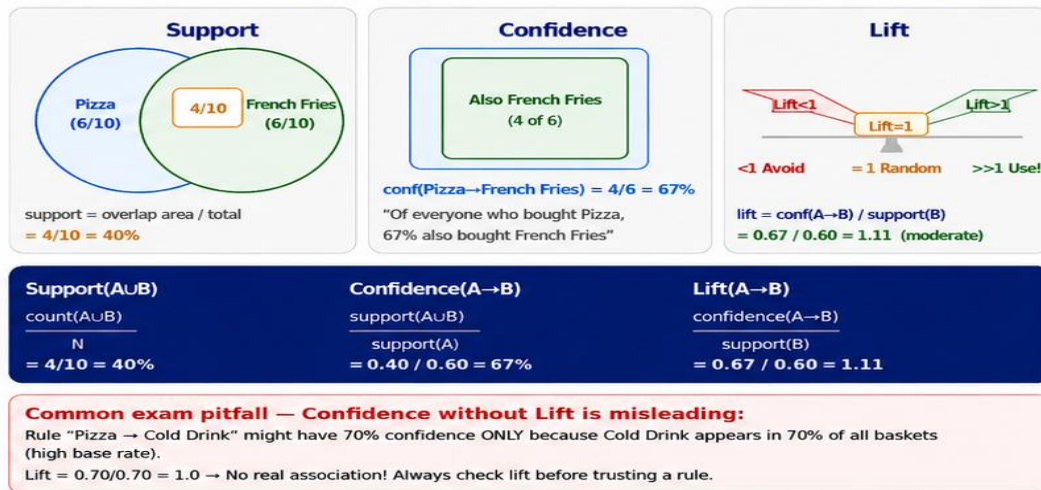
- $\text{conf}(\text{Pizza} \rightarrow \text{Cold Drink}) = 4/6 = 0.667$
- $\text{support}(\text{Cold Drink}) = 7/10 = 0.70$
- $\text{lift} = 0.667 / 0.70 = 0.95 \rightarrow$ Less than 1! Pizza buyers are actually **LESS** likely to buy Cold Drink than average. Despite the 67% confidence, this is **NOT** a useful rule.

Lift interpretation table:

Lift Value	Interpretation	Amazon Action
< 1	Negative association — A buyers LESS likely to buy B	Discard rule
= 1	No association — completely independent items	Discard rule
1.0 – 1.5	Weak positive association	Tablet — may vary by season
1.5 – 2.0	Moderate association	Test in recommendation widget

> 2.0	Strong positive association — genuine buying pattern	Promote in 'Frequently Bought Together'
-------	--	---

Visualising Support, Confidence, and Lift with Venn Diagrams



Section 4: The Apriori Algorithm — Full Step-by-Step Walkthrough

Apriori Algorithm — Step-by-Step Walkthrough

S1 Step 1: Scan database — count 1-itemsets

Pizza:6 Cold Drink:6 French Fries:6

MinSup = 40% = 4 transactions

Frequent: Pizza✓ Cold Drink✓ French Fries✓

S2 Step 2: Apriori pruning — key principle

If {French Fries} is infrequent, then {Pizza, French Fries}, {Cold Drink, French Fries}, and ALL supersets are automatically infrequent.

→ Do NOT generate or test those candidates. Huge saving!

S3 Step 3: Generate 2-item candidates from frequent 1-items

Candidates: {P,CD} {P,FF} {CD,FF}

Count each: {P,CD}=4 {P,FF}=4 {CD,FF}=3

{P,CD}=4 → kept {P,FF}=4 → kept {CD,FF}=3 → pruned (needs 4)
2 frequent 2-itemsets survive.

S4 Step 4: Extract rules from frequent itemsets

For each frequent pair {A,B} generate: A→B and B→A

Calculate: support, confidence, lift for each direction

Keep rules with conf ≥ 60% AND lift > 1.0 → Amazon promotion list

Item frequencies at a glance:

Pizza: 6

Cold Drink: 6

French Fries: 6

Eggs: 0

Cheese: 0

Yogurt: 0

4.1 The Core Principle

Apriori Principle (Anti-Monotonicity Property)

Statement: If an itemset is infrequent, then ALL of its supersets are also infrequent.

Equivalently: If an itemset IS frequent, then ALL of its subsets are also frequent.

Example: If {Pasta, Ice Cream} is infrequent (support < 40%), then {Pizza, Pasta, Ice Cream}, {Cold Drink, Pasta, Ice Cream}, and every other itemset containing both Pasta and Ice Cream is automatically infrequent. We never need to count them.

Why it matters: Without pruning, a dataset with 100 unique items has 2^{100} possible itemsets. The Apriori Principle cuts this down to a manageable subset.

4.2 Complete Algorithm with Our Example

Using the 10 grocery transactions, minSupport = 40%, minConfidence = 60%:

Pass 1 — Generate Frequent 1-Itemsets

1. Scan database: Count occurrences of each item across all 10 transactions.

2. Apply minSupport: Items with count < 4 are pruned. Ice Cream (3) is removed.
3. Result L1: {Pizza}=6, {Cold Drink}=7, {French Fries}=6, {Burger}=6, {Pasta}=4

Pass 2 — Generate Frequent 2-Itemsets

4. Generate candidates C2: All pairs from L1 = $C(5,2) = 10$ pairs. Note: if Ice Cream had been in L1 with k items, we'd have $C(k+1,2)$ more pairs — pruning it saves those tests.
5. Scan database: Count each pair. Requires one full database scan.
6. Apply minSupport: {Cold Drink,Burger}=3, {Cold Drink,Pasta}=3, {French Fries,Pasta}=2, {Burger,Pasta}=2, {Pizza,Pasta}=0 are all pruned.
7. Result L2: {Pizza,Cold Drink}=4, {Pizza,French Fries}=4, {Pizza,Burger}=4, {Cold Drink,French Fries}=4, {French Fries,Burger}=4

Pass 3 — Check for Frequent 3-Itemsets

8. Generate candidates C3: Combine pairs from L2 that share one item. e.g. {Pizza,Cold Drink}+{Pizza,French Fries} $\rightarrow$ {Pizza,Cold Drink,French Fries}.
9. Apriori pruning check: Before counting, verify every 2-item subset of the candidate is in L2. {Pizza,Cold Drink,French Fries}: subsets are {Pizza,Cold Drink}✓, {Pizza,French Fries}✓, {Cold Drink,French Fries}✓ — all in L2. Passes!
10. Scan database: {Pizza,Cold Drink,French Fries} appears in T01,T04,T07 = count 3. Support = 30% $< 40\%$ $\rightarrow$ pruned.
11. Result L3: Empty. No frequent 3-itemsets. Algorithm terminates.

4.3 Rule Generation from Frequent Itemsets

For each frequent k -itemset ($k \geq 2$), generate ALL possible rules by splitting into LHS and RHS:

```
For frequent itemset {A, B}:
  Rule 1: A  $\rightarrow$  B    (LHS={A}, RHS={B})
  Rule 2: B  $\rightarrow$  A    (LHS={B}, RHS={A})
```

For frequent itemset {Netflix, Amazon Prime Video, Disney+ Hotstar}:

```
Rule 1: A  $\rightarrow$  {Amazon Prime Video, Disney+ Hotstar}
Rule 2: B  $\rightarrow$  {Netflix, Disney+ Hotstar}
Rule 3: C  $\rightarrow$  {Netflix, Amazon Prime Video}
Rule 4: {Netflix, Amazon Prime Video}  $\rightarrow$  C
Rule 5: {Netflix, Disney+ Hotstar}  $\rightarrow$  B
Rule 6: {Amazon Prime Video, Disney+ Hotstar}  $\rightarrow$  A
```

Apply confidence filter: discard rules below minConfidence (60%)

Apply lift filter: discard rules with $\text{lift} \leq 1.0$

4.4 Time & Space Complexity

Step	Time Complexity	Space Complexity	Notes
Generate L_k	$O(L(k-1) ^2 \times N)$	$O(L_k)$	N = number of transactions
Scan for counts	$O(N \times C_k)$	$O(C_k)$	One full DB scan per pass
Total passes	$O(\text{max_k})$ scans	$O(L_1 + L_2 + \dots)$	max_k = length of longest frequent itemset
Rule generation	$O(2^k \times L_k)$	$O(\text{rules})$	Exponential in itemset size

Section 5: All Mined Rules — Confidence, Lift & Amazon Decision

Mined Association Rules — Confidence & Lift Summary						
Rule (A → B)	Support	Confidence	Lift	Rec?	Amazon Action	
Pizza → French Fries	40%	67%	1.11	NO	→ Monitor	
French Fries → Pizza	40%	67%	1.11	NO	→ Monitor	
Pizza → Cold Drink	40%	67%	1.11	NO	→ Monitor	
Cold Drink → Pizza	40%	67%	1.11	NO	→ Monitor	
Cold Drink → French Fries	30%	50%	0.83	NO	→ Discard	
French Fries → Cold Drink	30%	50%	0.83	NO	→ Discard	

Colour guide: High lift & conf → Amazon widget Moderate → monitor trends Lift ≤ 1 → negative/no association

5.1 Complete Rules Table for Grocery Example

Computing all rules from the 5 frequent 2-itemsets. MinConf = 60%. N = 10.

Rule (A → B)	Sup(AUB)	Sup(A)	Sup(B)	Confidence	Lift	Rec?
Pizza → French Fries	40% (4)	60% (6)	60% (6)	4/6 = 67%	0.67/0.60 = 1.11	MODERATE
French Fries → Pizza	40% (4)	60% (6)	60% (6)	4/6 = 67%	0.67/0.60 = 1.11	MODERATE
Pizza → Cold Drink	40% (4)	60% (6)	70% (7)	4/6 = 67%	0.67/0.70 = 0.95	NO — lift < 1
Cold Drink → Pizza	40% (4)	70% (7)	60% (6)	4/7 = 57%	0.57/0.60 = 0.95	NO — low conf
Pizza → Burger	40% (4)	60% (6)	60% (6)	4/6 = 67%	0.67/0.60 = 1.11	MODERATE
Burger → Pizza	40% (4)	60% (6)	60% (6)	4/6 = 67%	0.67/0.60 = 1.11	MODERATE
Cold Drink → French Fries	40% (4)	70% (7)	60% (6)	4/7 = 57%	0.57/0.60 = 0.95	NO — low conf

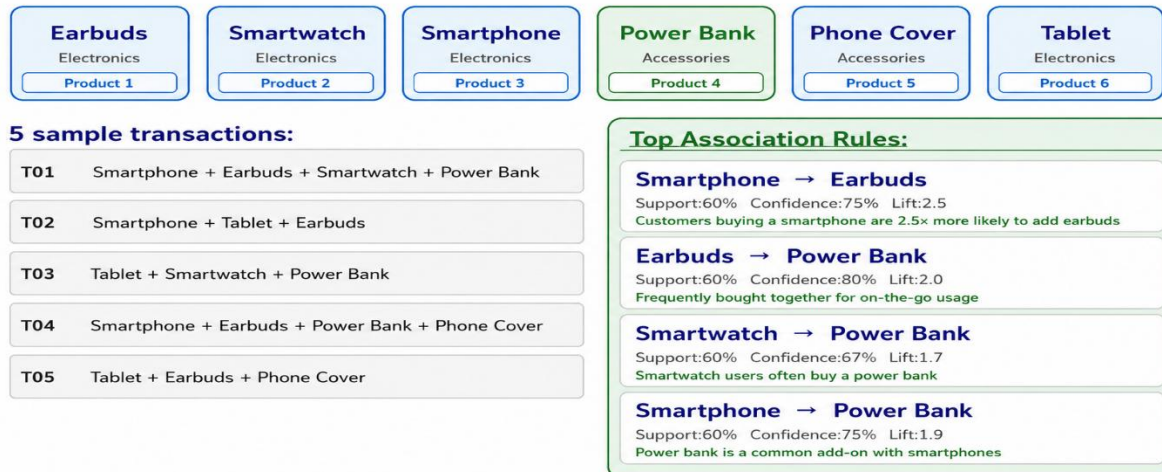
French Fries → Cold Drink	40% (4)	60% (6)	70% (7)	$4/6 = 67\%$	$0.67/0.70 = 0.95$	NO — lift < 1
French Fries → Burger	40% (4)	60% (6)	60% (6)	$4/6 = 67\%$	$0.67/0.60 = 1.11$	MODERATE
Burger → French Fries	40% (4)	60% (6)	60% (6)	$4/6 = 67\%$	$0.67/0.60 = 1.11$	MODERATE

Key Insight from this Example

The lift values in this grocery dataset are all close to 1.0. This is because we only have 10 transactions and 5 items are all approximately equally popular (60-70% support each). In a real Amazon dataset with millions of transactions and thousands of items, products have VERY different support levels — a niche item might have 0.01% support while a bestseller has 15%. Rules crossing popular-to-niche categories show high lift (e.g., 'Smartphone Case → Gaming Smartphone: lift=8.2') and are exactly what Amazon wants to surface.

Section 6: Practical Example 2 — Amazon Electronics (5 Transactions)

Practical Example 2 — Amazon Electronics Basket Analysis



Scenario: Amazon's electronics category team analyses 5 customer orders to identify product bundles. Items: Smartphone, Earbuds, Smartwatch, Power Bank, Phone Cover, Tablet. MinSupport = 40% (2/5), MinConfidence = 60%.

6.1 Transaction Database

Transaction	Items	Contains Smartphone?	Contains Earbuds?
E01	{Smartphone, Earbuds, Smartwatch, Power Bank}	YES	YES
E02	{Smartphone, Phone Cover, Earbuds}	YES	YES
E03	{Tablet, Smartwatch, Power Bank}	NO	NO
E04	{Smartphone, Earbuds, Power Bank, Phone Cover}	YES	YES

E05	{Tablet, Earbuds, Smartwatch}	NO	YES
-----	-------------------------------	----	-----

6.2 1-Itemset Frequencies

Item	Count	Support	Frequent ($\geq 40\%$)?
Earbuds	4/5	80%	YES ✓
Smartwatch	3/5	60%	YES ✓
Smartphone	3/5	60%	YES ✓
Power Bank	3/5	60%	YES ✓
Phone Cover	2/5	40%	YES ✓
Tablet	2/5	40%	YES ✓

6.3 Top Rules with High Lift

Rule	Support	Confidence	Lift	Business Meaning
Smartphone → Earbuds	3/5 = 60%	3/3 = 100%	$100\%/80\% = 1.25$	Every laptop buyer also buys a mouse
Earbuds → Smartphone	3/5 = 60%	3/4 = 75%	$75\%/60\% = 1.25$	Most mouse buyers are laptop buyers
Smartphone → Power Bank	3/5 = 60%	3/3 = 100%	$100\%/60\% = 1.67$	Smartphone buyers strongly prefer Power Banks
Phone Cover → Smartphone	2/5 = 40%	2/2 = 100%	$100\%/60\% = 1.67$	Phone Cover buyers always also bought a laptop
Smartwatch → Power Bank	2/5 = 40%	2/3 = 67%	$67\%/60\% = 1.11$	Moderate keyboard-hub association

With only 5 transactions, the lift values are moderate (1.1–1.67). Amazon in production uses hundreds of millions of transactions. Rules like 'Smartphone → Smartphone Case: lift=9.2' or 'DSLR Camera → SD Card: lift=7.8' emerge clearly from the noise because one product strongly predicts the other — these become the 'Frequently Bought Together' recommendations.

Amazon also applies TIME WINDOWS — rules are recomputed weekly or daily so that seasonal patterns (winter → warm clothing) and trend shifts (new product launches) are captured. Static rules computed once become stale.

Section 7: Practical Example 3 — Manual Apriori (Exam-Style)

Exam scenario: This is the type of question that appears in BCA Data Mining exams. A 6-transaction database is given. Apply Apriori manually with $\text{minSupport} = 50\%$ ($3/6$) and $\text{minConfidence} = 75\%$. Show all steps.

7.1 The Given Database

TID	Items
100	{Netflix, Disney+ Hotstar, Sony LIV}
200	{Amazon Prime Video, Disney+ Hotstar, JioCinema}
300	{Netflix, Amazon Prime Video, Disney+ Hotstar, JioCinema}
400	{Amazon Prime Video, JioCinema}
500	{Netflix, Amazon Prime Video, Disney+ Hotstar, JioCinema}
600	{Netflix, Amazon Prime Video, Disney+ Hotstar}

7.2 Pass 1 — Frequent 1-Itemsets

Item	Count	Support	$\geq 50\%$?
A	100,300,500,600 = 4	$4/6 = 67\%$	YES ✓
B	200,300,400,500,600 = 5	$5/6 = 83\%$	YES ✓
C	100,200,300,500,600 = 5	$5/6 = 83\%$	YES ✓
D	100 = 1	$1/6 = 17\%$	NO — pruned X

E	200,300,400,500 = 4	4/6 = 67%	YES ✓
---	---------------------	-----------	-------

L1 = {A}, {B}, {C}, {E} (D is pruned — no superset of D needs checking)

7.3 Pass 2 — Frequent 2-Itemsets

C2 = all pairs from {A,B,C,E} = {Netflix, Amazon Prime Video}, {Netflix, Disney+ Hotstar}, {A,E}, {Amazon Prime Video, Disney+ Hotstar}, {Amazon Prime Video, JioCinema}, {Disney+ Hotstar, JioCinema}

2-Itemset	Transactions	Count	Support	≥ 50%?
{A, B}	300,500,600	3	3/6 = 50%	YES ✓
{A, C}	100,300,500,600	4	4/6 = 67%	YES ✓
{A, E}	300,500	2	2/6 = 33%	NO ✗
{B, C}	200,300,500,600	4	4/6 = 67%	YES ✓
{Amazon Prime Video, JioCinema}	200,300,400,500	4	4/6 = 67%	YES ✓
{C, E}	200,300,500	3	3/6 = 50%	YES ✓

L2 = {Netflix, Amazon Prime Video}, {Netflix, Disney+ Hotstar}, {Amazon Prime Video, Disney+ Hotstar}, {Amazon Prime Video, JioCinema}, {Disney+ Hotstar, JioCinema}

7.4 Pass 3 — Frequent 3-Itemsets

Generate **C3** from **L2**. Apriori pruning: every 2-item subset of each candidate must be in **L2**.

Candidate	2-item subsets	All in L2?	Count	Support	≥ 50%?
{Netflix, Amazon Prime Video, Disney+ Hotstar}	{Netflix, Amazon Prime Video}✓ {Netflix, Disney+ Hotstar}✓ {Amazon Prime Video, Disney+ Hotstar}	YES	300,500,600 = 3	50%	YES ✓

Disney+ Hotstar}	Prime Video, Disney+ Hotstar}✓				
{A,B,E}	{Netflix, Amazon Prime Video}✓ {A,E}X {Amazon Prime Video, JioCinema}✓	NO — pruned	—	—	X
{A,C,E}	{Netflix, Disney+ Hotstar}✓ {A,E}X {Disney+ Hotstar, JioCinema}✓	NO — pruned	—	—	X
{Amazon Prime Video, Disney+ Hotstar, JioCinema}	{Amazon Prime Video, Disney+ Hotstar}✓ {Amazon Prime Video, JioCinema}✓ {Disney+ Hotstar, JioCinema}✓	YES	200,300,500 = 3	50%	YES ✓

L3 = {Netflix, Amazon Prime Video, Disney+ Hotstar}, {Amazon Prime Video, Disney+ Hotstar, JioCinema}

7.5 Rule Generation (MinConf = 75%)

From {Netflix, Amazon Prime Video, Disney+ Hotstar} and {Amazon Prime Video, Disney+ Hotstar, JioCinema} generate all directional rules:

Rule	Confidence Calculation	Confidence	≥ 75%?
A,B → C	$\text{support}(\{\text{Netflix, Amazon Prime Video, Disney+ Hotstar}\}) / \text{support}(\{\text{Netflix, Amazon Prime Video}\}) = 0.50/0.50 = 1.00$	100%	YES ✓
A,C → B	$0.50 / 0.67 = 0.75$	75%	YES ✓
B,C → A	$0.50 / 0.67 = 0.75$	75%	YES ✓

$A \rightarrow B, C$	$0.50 / 0.67 = 0.75$	75%	YES ✓
$B \rightarrow A, C$	$0.50 / 0.83 = 0.60$	60%	NO ✗
$C \rightarrow A, B$	$0.50 / 0.83 = 0.60$	60%	NO ✗
$B, C \rightarrow E$	$0.50 / 0.67 = 0.75$	75%	YES ✓
$B, E \rightarrow C$	$0.50 / 0.67 = 0.75$	75%	YES ✓
$C, E \rightarrow B$	$0.50 / 0.50 = 1.00$	100%	YES ✓
$B \rightarrow C, E$	$0.50 / 0.83 = 0.60$	60%	NO ✗

Final Answer for This Exam Question

Frequent itemsets: $L1=\{A\},\{B\},\{C\},\{E\}$ | $L2=\{\text{Netflix, Amazon Prime Video}\},\{\text{Netflix, Disney+ Hotstar}\},\{\text{Amazon Prime Video, Disney+ Hotstar}\},\{\text{Amazon Prime Video, JioCinema}\},\{\text{Disney+ Hotstar, JioCinema}\}$ | $L3=\{\text{Netflix, Amazon Prime Video, Disney+ Hotstar}\},\{\text{Amazon Prime Video, Disney+ Hotstar, JioCinema}\}$

Strong rules ($\text{conf} \geq 75\%$): $\{\text{Netflix, Amazon Prime Video}\} \rightarrow C$, $\{\text{Netflix, Disney+ Hotstar}\} \rightarrow B$, $\{\text{Amazon Prime Video, Disney+ Hotstar}\} \rightarrow A$, $A \rightarrow \{\text{Amazon Prime Video, Disney+ Hotstar}\}$, $\{\text{Amazon Prime Video, Disney+ Hotstar}\} \rightarrow E$, $\{\text{Amazon Prime Video, JioCinema}\} \rightarrow C$, $\{\text{Disney+ Hotstar, JioCinema}\} \rightarrow B$

Exam tip: Always show the support calculation step-by-step. Write 'pruned by Apriori principle' when skipping a candidate — this demonstrates understanding of the algorithm, not just the formula.

Section 8: FP-Growth vs Apriori — Comparison

Context: Apriori requires multiple database scans (one per itemset length level k). For a dataset like Amazon's with millions of items and transactions, this is impractically slow. FP-Growth was developed to solve this with only 2 database scans.

8.1 FP-Tree Construction

FP-Growth compresses the transaction database into a compact tree structure called an FP-Tree:

12. **Scan 1:** Count all item frequencies. Sort frequent items by frequency (descending). Prune infrequent items.
13. **Scan 2:** For each transaction, sort items by their frequency rank and insert into the tree. Shared prefixes are merged into a single path, saving space.
14. **Pattern extraction:** For each frequent item (starting from the least frequent), trace its conditional pattern base — all paths leading to it in the tree. Build a conditional FP-Tree and recursively mine it.

8.2 FP-Tree for Our Grocery Example

Sorted item order: Cold Drink(7) > Pizza(6) = French Fries(6) = Burger(6) > Pasta(4)

FP-Tree after inserting all 10 transactions:

```

ROOT
├── Cold Drink:6
│   ├── Pizza:4
│   │   ├── French Fries:3 ← T01,T04,T07
│   │   │   └── Burger:2
│   │   └── Burger:1 ← T09
│   └── French Fries:1
│       └── Pasta:1 ← T10
└── Pizza:2 (not preceded by Cold Drink)
    ├── French Fries:1 ← T02
    └── Burger:1
  
```

Conditional pattern base for Burger:

```

{Cold Drink,Pizza,French Fries}: 2 | {Cold Drink,Pizza}: 1 |
{Pizza,French Fries}: 1 | {Cold Drink,French Fries}: 1
  
```

8.3 Comparison Table

Feature	Apriori	FP-Growth
Database scans required	k scans (one per itemset level)	Exactly 2 scans
Candidate generation	Explicit — generates all Ck candidates	None — pattern base extracted from tree
Memory usage	Low (stores only candidate counts)	Higher (FP-Tree stored in RAM)
Speed on sparse data	Fast (few frequent itemsets)	Overkill for sparse data
Speed on dense data	Very slow	10-100× faster than Apriori
Implementation complexity	Simple	Moderately complex (tree construction)
Scalability	Struggles with millions of items	Handles billion-transaction datasets
Amazon production use	Not used at scale	FP-Growth variant used

Section 9: Industry Application — How Amazon Uses Association Rules

The Scale

- **Transactions per day:** Amazon processes ~13 million orders per day globally (peak days: 40M+)
- **Items in catalog:** 350+ million unique products
- **Rules computed:** Hundreds of millions of product-pair rules are maintained
- **Update frequency:** Rules are recomputed daily using distributed Spark/MapReduce jobs
- **Revenue impact:** Amazon attributes 35% of total revenue to recommendation systems

9.1 Where Association Rules Appear on Amazon

Feature	Association Rule Application	Typical Lift Range
'Frequently Bought Together' widget	High-confidence, high-lift product pairs shown on product page	3.0 – 15.0
'Customers who viewed X also viewed'	Web-usage mining rules: view-to-view associations	1.5 – 5.0
Bundle pricing	Rules identify natural bundles for discount pricing	2.0 – 8.0
Search result ordering	Products with high association to query terms ranked higher	1.2 – 3.0
Email recommendations	Post-purchase rules: 'You bought X, you may want Y'	2.0 – 10.0
Inventory placement	Co-purchased items stored in same warehouse (reduces shipping cost)	4.0+

9.2 The Full Amazon Pipeline

Amazon's association rule pipeline involves several stages beyond basic Apriori/FP-Growth:

15. **Data collection:** Every click, view, add-to-cart, and purchase is logged with user ID, session ID, timestamp, and product ID.
16. **Session construction:** Individual events are grouped into 'sessions' (typically a 30-minute window). A session is treated as one 'transaction'.
17. **Distributed FP-Growth:** The transaction database is partitioned across thousands of servers. Each server computes local FP-Trees. Results are merged to produce global frequent itemsets.
18. **Rule filtering:** Rules are filtered by minSupport (varies by category), minConfidence (60%+), and minLift (1.5+). Category-crossing rules and rules involving the same brand are handled separately.
19. **A/B testing:** New rules are not immediately shown to all users. They are tested on a small percentage of traffic. Click-through rate and conversion rate determine whether they graduate to full deployment.
20. **Contextual weighting:** Rules are further filtered by user context — a user browsing baby products gets baby-relevant rules even if they have a general high lift. Personalisation layer sits on top of association rules.

Section 10: Summary — Formulas, Pitfalls & Exam Tips

10.1 Complete Formula Reference

Metric	Formula	Range	What we want
Support	$\text{count}(A \cup B) / N$	0 to 1	Higher = more frequent
Confidence	$\text{support}(A \cup B) / \text{support}(A)$	0 to 1	Higher = more reliable
Lift	$\text{confidence}(A \rightarrow B) / \text{support}(B)$	> 0	Higher than 1 = genuine association
Conviction	$(1 - \text{support}(B)) / (1 - \text{confidence}(A \rightarrow B))$	1 to ∞	High = strong directional rule

10.2 Apriori Algorithm Summary

```

APRIORI(D, minSup, minConf):
    L1 = frequent 1-itemsets from D
    k = 2
    while Lk-1 is not empty:
        Ck = apriori_gen(Lk-1)           // generate k-item candidates
        for each transaction t in D:      // one database scan
            Ct = subset(Ck, t)           // candidates contained in t
            for each c in Ct: c.count++
        Lk = {c in Ck | c.count >= minSup * N}
        k++
    L = union of all Lk
    Rules = genRules(L, minConf)         // generate and filter rules
    return Rules

```

10.3 Common Exam Questions & Answers

21. Q: What does the Apriori principle state? A: If an itemset is infrequent, all its supersets are also infrequent. This allows candidate pruning without counting.

22. Q: Why is lift more important than confidence? A: Confidence alone is misleading for popular items. If Cold Drink appears in 70% of baskets, any rule $X \rightarrow \text{Cold Drink}$ achieves ~70% confidence automatically. Lift divides by $\text{support}(\text{Cold Drink})$ to correct for this baseline, revealing the true association strength.
23. Q: What is the difference between support and confidence? A: Support measures how often {Netflix, Amazon Prime Video} appears together (absolute frequency). Confidence measures how often B appears given A (conditional probability). Confidence is directional; support is symmetric.
24. Q: How many database scans does Apriori require? A: k scans where k is the length of the longest frequent itemset. Each scan generates one level of frequent itemsets.
25. Q: Why does FP-Growth need only 2 database scans? A: Scan 1 builds the frequency-sorted header table. Scan 2 constructs the FP-Tree. After that, all pattern mining is done on the in-memory tree — no further database access is needed.
26. Q: Give two real-world applications of association rules beyond e-commerce. A: (1) Medical diagnosis: co-occurring symptoms (fever + cough + fatigue $\rightarrow$ flu) help triage systems. (2) Web click-stream analysis: pages visited together help with content recommendation and site navigation design.

Common Mistakes — Do NOT Do This in Exams

- Mistake 1: Confusing confidence and lift. Confidence measures reliability; lift measures strength vs random. A rule can have high confidence but lift = 1 (useless). Always compute both.
- Mistake 2: Forgetting to check the Apriori pruning condition when generating 3-itemsets. Every 2-item subset of a 3-item candidate must be in L2 before you count it.
- Mistake 3: Computing support as count/total transactions, then forgetting to compare with the MINIMUM support COUNT (not percentage) when the question gives 'minSup = 3 transactions'.
- Mistake 4: Assuming $\text{confidence}(A \rightarrow B) = \text{confidence}(B \rightarrow A)$. These are NOT equal unless $\text{support}(A) = \text{support}(B)$.
- Mistake 5: Treating all rules with confidence $\geq$ threshold as 'strong' without checking lift. Lift < 1 means the rule is actually a NEGATIVE association — showing it as a recommendation would decrease sales.

Quick Reference — All Key Values for Grocery Example

Transactions: $N = 10$ | MinSupport = 40% (= 4 transactions) | MinConfidence = 60%

Frequent 1-itemsets: Pizza(6), Cold Drink(7), French Fries(6), Burger(6), Pasta(4) | Pruned: Ice Cream(3)

Frequent 2-itemsets: {Pizza,Cold Drink}=4, {Pizza,French Fries}=4, {Pizza,Burger}=4, {Cold Drink,French Fries}=4, {French Fries,Burger}=4

Pruned 2-itemsets: {Cold Drink,Burger}=3, {Cold Drink,Pasta}=3, {French Fries,Pasta}=2, {Burger,Pasta}=2, {Pizza,Pasta}=0

Frequent 3-itemsets: None — all 3-item candidates fall below 40% support

Key formula check: $\text{lift}(\text{Pizza} \rightarrow \text{French Fries}) = \frac{\text{conf}(\text{French Fries})}{\text{sup}(\text{French Fries})} = \frac{(4/6)}{(6/10)} = 0.667/0.60 = 1.11$

Write a Program to Demonstrate Apriori Algorithm.

Source Code:

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules


def getch():
    input("\nPress Enter to continue...")


transactions = []

df = None

frequent_itemsets = None


def load_excel():
    global transactions

    file_path = "midata.xlsx"
    # file_path = input("Enter Excel file path (e.g., data.xlsx): ")

    try:

        data = pd.read_excel(file_path, engine='openpyxl')

        print("\nExcel Data Loaded Successfully!")

        print(data.head())
```

```
transactions.clear()
```

```
for index, row in data.iterrows():
```

```
    items = [str(item) for item in row if pd.notna(item)]
```

```
    transactions.append(items)
```

```
print("\nTransactions Created Successfully!")
```

```
except Exception as e:
```

```
    print("Error loading file:", e)
```

```
def show_transactions():
```

```
    if not transactions:
```

```
        print("\n⚠ No data loaded!")
```

```
    return
```

```
print("\n--- Transactions ---")
```

```
for i, t in enumerate(transactions, start=1):
```

```
    print(f"T{i}: {t}")
```

```
def convert_to_dataframe():
```

```
    global df
```

```
    if not transactions:
```

```
        print("\n⚠ Load Excel data first!")
```

```
    return
```

```
te = TransactionEncoder()
```

```
te_data = te.fit(transactions).transform(transactions)

df = pd.DataFrame(te_data, columns=te.columns_)

print("\n--- One-Hot Encoded Data ---")
print(df)

def run_apriori():
    global df, frequent_itemsets

    if df is None:
        print("\n⚠ Convert data first!")
        return

    min_sup = float(input("Enter minimum support (e.g., 0.3): "))

    frequent_itemsets = apriori(df, min_support=min_sup, use_colnames=True)

    print("\n--- Frequent Itemsets ---")
    print(frequent_itemsets)

def generate_rules():
    global frequent_itemsets

    if frequent_itemsets is None or frequent_itemsets.empty:
        print("\n⚠ Run Apriori first and ensure itemsets are generated!")
        return

    try:
        min_conf = float(input("Enter minimum confidence (e.g., 0.6): "))
```

```
rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=min_conf)

if rules.empty:

    print("\n⚠ No rules found! Try lowering support or confidence.")
else:

    print("\n--- Association Rules ---")

    # Convert frozenset to readable list
    rules['antecedents'] = rules['antecedents'].apply(lambda x: list(x))
    rules['consequents'] = rules['consequents'].apply(lambda x: list(x))

    print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])

except Exception as e:

    print("Error generating rules:", e)

while True:

    print("\n=====")
    print(" APRIORI ALGORITHM (EXCEL)")
    print("=====")
    print("1. Load Data from Excel")
    print("2. Show Transactions")
    print("3. Convert to One-Hot Data")
    print("4. Run Apriori Algorithm")
    print("5. Generate Association Rules")
    print("6. Exit")

    choice = input("Enter your choice: ")
```

```
if choice == '1':
    load_excel()
    getch()

elif choice == '2':
    show_transactions()
    getch()

elif choice == '3':
    convert_to_dataframe()
    getch()

elif choice == '4':
    run_apriori()
    getch()

elif choice == '5':
    generate_rules()
    getch()

elif choice == '6':
    print("Exiting program...")
    break

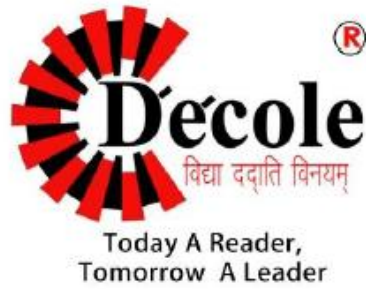
else:
    print("Invalid choice!")
    getch()
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\tanwa\Desktop\p> py datamining.py

=====
APRIORI ALGORITHM (EXCEL)
=====
1. Load Data from Excel
2. Show Transactions
3. Convert to One-Hot Data
4. Run Apriori Algorithm
5. Generate Association Rules
6. Exit
Enter your choice: 1

Excel Data Loaded Successfully!
  Item1      Item2      Item3      Item4
0 Smartphone Earbuds Smartwatch Power Bank
1 Smartphone Phone Cover Earbuds NaN
2 Tablet Smartwatch Power Bank NaN
3 Smartphone Earbuds Power Bank Phone Cover
4 Tablet Earbuds Smartwatch NaN

Transactions Created Successfully!
```



Dezyne École College
www.dezyneecole.com

Thank You